Report on the second version of the firmware board analyzer of the device manufacturer

Engineer Samira Sadat Jalali Parvin

Prepared for

Sedna Company

Summer 2025

Abstract

In patient oxygen analyzers, a precise and intelligent monitoring system plays a vital role in ensuring the health and safety of the patient. This system continuously measures the amount of oxygen emitted from the device and activates a warning alarm in time if any deviation occurs from the permissible range. Rapid detection of an abnormal drop or increase in oxygen levels allows the medical team or the user to take corrective action in the shortest possible time and prevent possible .risks

From a technical standpoint, the device is designed for ultra-low power consumption, ensuring stable operation even during prolonged use or when running on emergency power (battery or backup source). Intelligent energy management ensures that the measurement and display components remain active only when needed, switching to a low-power (sleep) mode during periods of .inactivity

Table of contents

page **title**

Chapter 1: Introduction to the Project Overview

This project involves implementing the firmware for an oxygen concentrator analyzer board designed to acquire data from an oxygen sensor and display it on a 2.4-inch OLED screen.

The OLED display is oriented vertically, presenting sensor data—specifically oxygen purity, airflow, and air temperature—which constitute the project's three primary parameters.

Another key feature is the menu system, which allows the user to set custom thresholds for these three parameters; if the readings exceed these limits, a buzzer triggers an alarm. The OLED consists of four pages. The first page displays the logo and the initial setup of the device, the second page displays the sensor information that is always done, the third page contains the menu options, and the fourth page determines the allowable limit for the user's desired option.

The initial setup of the device is as follows: by pressing the on button, the desired voltage is applied to the microcontroller, and the microcontroller, while displaying the first page of the OLED, reads the information stored in the non-volatile flash memory. This information includes the previously set and saved limits on the microcontroller.

There are two methods for turning off the device. The first method is to press the off button, after which the logo page is displayed and all the required information is stored, and the microcontroller itself cuts off its power circuit and turns off. The second method occurs if the oxygen purity value has not changed for a certain period of time, in which case the device will also turn itself off.

Chapter 2 - Introduction to the different parts of hardware

Principles of button operation

The button installed on the left serves as the microcontroller's on/off (latching) switch. Based on the circuit design, the firmware must be programmed so that the microcontroller can use a specific output pin to maintain its own 3.3V power supply. The implementation works as follows: pressing the button supplies 3.3V to the microcontroller, while releasing it cuts off the power. Upon receiving this 3.3V, the microcontroller sets the designated wake-up output pin to a high state (logic 1); as long as this voltage remains high, the microcontroller stays powered on, but if the pin goes low, the microcontroller shuts itself down.

The middle button, called the menu button, receives commands from the user as analog input to control movement between menu pages or to select and apply an option.

Two up and down buttons are provided for moving between menu options or increasing and decreasing parameter values. These two buttons are defined as interrupts for better and faster reception .

Principles of OLED work

According to the request, the OLED screen must be vertical, unlike all OLEDs that are designed as 64*128. And its information refresh must change very smoothly and there is no need to clear the entire screen and resend the information buffer. The brightness of the OLED screen can also be adjusted by the user.

On the second page, while displaying the main parameter values, the battery charge value is displayed at the top of the page and blinks if its charge is less than 20 percent.

Codes related to battery blinking during timer interruption:

```
BatteryRefresh++;
if(Vbat<2700) BlinkBattery=0;
else  BlinkBattery=1;
if (BatteryRefresh>3) BatteryRefresh=0;
```

Next to the battery symbol, there is a buzzer symbol that indicates whether the alarm is on or not. This feature is also determined by the user whether the device will give an alarm or not.

In this version, the OLED works with i2c communication, the challenges of which will be explained later in the project.

Working principles of the oxygen sensor

The connection between the oxygen sensor and the microcontroller is via UART. It works with a baud rate of 9600. Contrary to the information given in the datasheet of this module, this sensor can be started with 5 volts and the information can be received correctly. The sensor sends data every few milliseconds, which we always receive in the main loop of the program, and according to the protocol standard given for this sensor in the datasheet, we retrieve the data and extract the information. The information is received in the form of a hexadecimal number, which we convert to a decimal number and then display. **static const uint8_t** HeadarOCFS[] = {0x16, 0x9, 0x1};

If the received data is not sent with this header, it implies the data is corrupted or lost; therefore, it is not processed.

Working principles of the battery voltage reading circuit

To read the battery charge voltage, we need to get the voltage value from the relevant circuit using the ADC. The algorithm of this part is written in such a way that, considering the change in the value of the relevant base voltage in different conditions such as turning the buzzer on and off or any other current drop, the ADC value is read and sampled every 20 seconds in the main loop of the micro and the average of the samples is obtained. Then, according to the calculations required to convert this voltage to battery voltage, the percentage of charge is obtained

```c
uint32_t ReadBattery_mV() {
    ADC_ChannelConfTypeDef sConfig = {0};
    sConfig.Channel = 6;
     sConfig.Rank = 1;
    HAL_ADC_ConfigChannel(&hadc, &sConfig);

    HAL_ADC_Start(&hadc);
    HAL_ADC_PollForConversion(&hadc, HAL_MAX_DELAY);
    uint32_t ADC_Value = HAL_ADC_GetValue(&hadc);
    HAL_ADC_Stop(&hadc);

    uint32_t VDD = 3300;
    uint32_t Vadc_mV = (ADC_Value * VDD) / 4095; // مقدرات
    uint32_t Vbat_mV = (Vadc_mV) * ((R2 + R3)) / R3 +200;

    return (uint32_t)Vbat_mV;
}
void ShowBatteryPercentage() {

    if(inSettingsMenu==0){

        uint32_t Sum=0;

        for (int i =0 ; i<10 ; i++){
            Sum+=ReadBattery_mV();
        }
        Vbat=Sum/10;
```

Buzzer working principles

The buzzer turns on for 600 milliseconds and turns off for 600 milliseconds, or beeps, provided that the parameter limit is exceeded and the device is in unMute mode and is on the second OLED screen.

These conditions are checked in a timer that counts every 300 milliseconds, so that there is no delay in the main loop and that the sensor information is always received correctly.

```c
if(((ox_Concent<purityLow && purityLow>=700 && purityLow<=950) || ( ox_Flow>flowLow && flowLow<=120 && flowLow>=10) || (ox_Temperature>maxTemp && maxT
    if (beep==true){
        if (Beep==true){__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4, htim3.Init.Period / 2);Beep=false;}
        else if (Beep==false){__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4, 0);Beep=true;}
    }

    BuzzToggleTime=0;
}
else  if (ox_Concent>purityLow || ox_Flow<flowLow || ox_Temperature<maxTemp ) __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4, 0);
```

In the timer interrupt function, a condition is written that if the purity is less than the specified limit, the flow is greater than the specified limit, and the temperature is greater than the specified value, the buzzer will start beeping.

Chapter 3 – Software

Introduction to the programming environment

Mcu is stm32f and its programmer is stlink.

Programming in stmcube ide

Micro configuration

In Cube MX, after selecting the appropriate micro, the following items are selected in order:

- I2c for OLED communication with DMA
- Usart1 with DMA for oxygen sensor
- Adc and selecting the appropriate channel for reading battery voltage with 12-bit resolution and sampling time = 239.5 cycle
- The two interrupt pins mentioned and the input and output pins that were explained

Main.c file

Most tasks are controlled by timer .

```
90  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
91  {
92      if(htim->Instance == TIM1) // 300 بار ms
93      {
94        BuzzToggleTime++;
95        if(((ox_Concent<purityLow && purityLow>=700 && purityLow<=1000) || (ox_Flow<flowLow && fl
96              if (isMute==false) HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_0);
97              BuzzToggleTime=0;
98        }
99
100
101       if(ClickToOFF==true)  button_press_time++;
102       else button_press_time=0;
103
104       BatteryRefresh++;
105       if(Percent<35) BlinkBattery=0;
106       else  BlinkBattery=1;
107      if (BatteryRefresh>3) BatteryRefresh=0;
108      }
109 }
```

Designing 4 page for lcd display. In a State Machine, the program is in one state at any given moment and moves to another state when an event occurs.

```
641     if (Refresh_display==true){ // این شرط برای این است که هر بار در لوپ اصلی صفحه را رفرش نکند و فقط وقتی که تغییری حاصل شد صفحه رفرش شود
642         ssd1306_setFixedFont(ssd1306xled_font8x16);
643         ssd1306_printFixed2(110,0,SettingFrame, STYLE_NORMAL,0);
644         ssd1306_setFixedFont(ssd1306xled_font6x8);
645         for (int idx = 0; idx < MENU_COUNT; idx++) {
646             int y =12+ idx * 12;
647             if (idx == CurrentMenu && idx != 6) ssd1306_printFixed2(95-y,0,"*", STYLE_NORMAL,0);
648             else if(idx != CurrentMenu)ssd1306_printFixed2(95-y,0," ", STYLE_NORMAL,0);
649             else if (idx == 6) ssd1306_printFixed2(0,20,"*", STYLE_NORMAL,0);
650             if (idx != 6) ssd1306_printFixed2(0,20," ", STYLE_NORMAL,0);
651             ssd1306_drawBitmap (43,6,16,16,PixelClean);
652             if(idx != 6)ssd1306_printFixed2(95-y,10,(char*)items[idx], STYLE_NORMAL,0);
653             ssd1306_drawBitmap(0,3,7,25,bitmap_Back);
654
655       }
```

Notice : Additional file descriptions have been removed due to NDA agreements and confidentiality issues

File parameters_Val.h

This file is written for modular and functional programming.

File iconBits.h

This header file is for storing bits of images and Legos.

```
#ifndef INC_ICONSBIT_H_
#define INC_ICONSBIT_H_
void ShowLoadingScreen(uint8_t percent);
void ShowPage1 ();
const uint8_t SednaLogo[4095]={
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x80, 0x00, 0x00, 0x00, 0x0
        0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
        0xc0, 0x60, 0x10, 0xc8, 0xf4, 0xfa, 0xfc, 0x7f, 0x1f, 0x0f, 0x03, 0x00, 0xf0, 0x08, 0xe
        0xf4, 0xf4, 0xf8, 0xf0, 0x00, 0x03, 0x0f, 0x1f, 0x7f, 0xfe, 0xfe, 0xfc, 0xf8, 0xf0, 0xe
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0
```

Description of the Oled Library

There are three libraries available for OLED programming:

1.ssd1306: The famous OLED library. The problem with this library for this project is that the OLED library is very large. Because in order to send the data buffer to the OLED, it first fills the 128x64 buffer, which takes up about 1024 bits, or 1 kilobyte of memory. To make the image vertical, it rotates each character 90 degrees, then sends the bytes related to it to a new buffer to be sent. And this operation takes up another 1 kilobyte. The micro has 4 kilobytes of flash memory, which causes a stack overflow.

The second problem with this library is that because it sends the entire 1024-byte buffer every time it sends, for every small change in the page, the entire page must be refreshed, which causes a jump in the screen.

2.u8g9: A small and bulky library that also has the previous problems along with the complexity of the implementation.

3.alexy: The main difference between this library and other libraries is that it sends the same character every time it is created. This difference does not have a memory problem and does not cause a jump in the page. We only need to add the library to the program drivers section and give the path address to the software.

In this version, the OLED communication board is in SPI format, so the main requirement for this project will be to convert the OLED communications in the existing libraries to SPI.

Chapter Four – Challenges

Converting the library for communication

To convert the communication implementation of the existing library—which relied on a pre-I2C version—we need to apply this change across various sections of the library.

Fixing the buzzer sound issue

The newly installed buzzer is an SMD component that operates via a PWM signal. Therefore, we configure a PWM channel on the designated pin and adjust the pulse frequency to match the buzzer's resonant frequency, thereby generating the beep sound.

Fixed issues with the Old library regarding vertical display

To enable vertical display, a few functions must be added to the libraries; these are described below:

The `void ssd1306_printFixed2_SPI_6_8` function is an example designed to display 6x8 characters; it first swaps the X and Y coordinates—adjusting them according to the OLED command requirements—stores the result in a new array, and then transmits the data via SPI. Key aspects of this process include the character conversion sequence, the transmission order, and the correct, appropriate use of command codes.

With this mechanism in place, a text string can be transmitted character by character using the same function.

For 8x16 fonts, the function treats each character as two 8x8 characters, performing the necessary conversions and transmission sequencing accordingly.

Fixed bugs in the OLED library for displaying in different fonts

The functions implemented in `ssd1306_spi_Stm32.h` are designed for SPI data transmission, as the original library had flaws in this regard. The general operating principle is to first prepare the OLED for command or data transmission using the CS and DC pins, and then transmit the data via HAL libraries.

Data is transmitted byte-by-byte to avoid flash memory issues

```
// ------------------------------------------------
// ◇ Initialization for STM32
// ------------------------------------------------

void ssd1306_WriteCommand(uint8_t cmd)
{
    OLED_DC_LOW();
    // OLED_CS_LOW();               // DC=0 → فرمان
    HAL_SPI_Transmit(&hspi1, &cmd, 1, HAL_MAX_DELAY);

}

void ssd1306_WriteData(uint8_t *data, uint16_t size)
{
    OLED_DC_HIGH();
    OLED_CS_LOW();
    HAL_SPI_Transmit(&hspi1, data, size, HAL_MAX_DELAY);
    // OLED_CS_HIGH();
}
```

Fixed an issue with reading adc values correctly

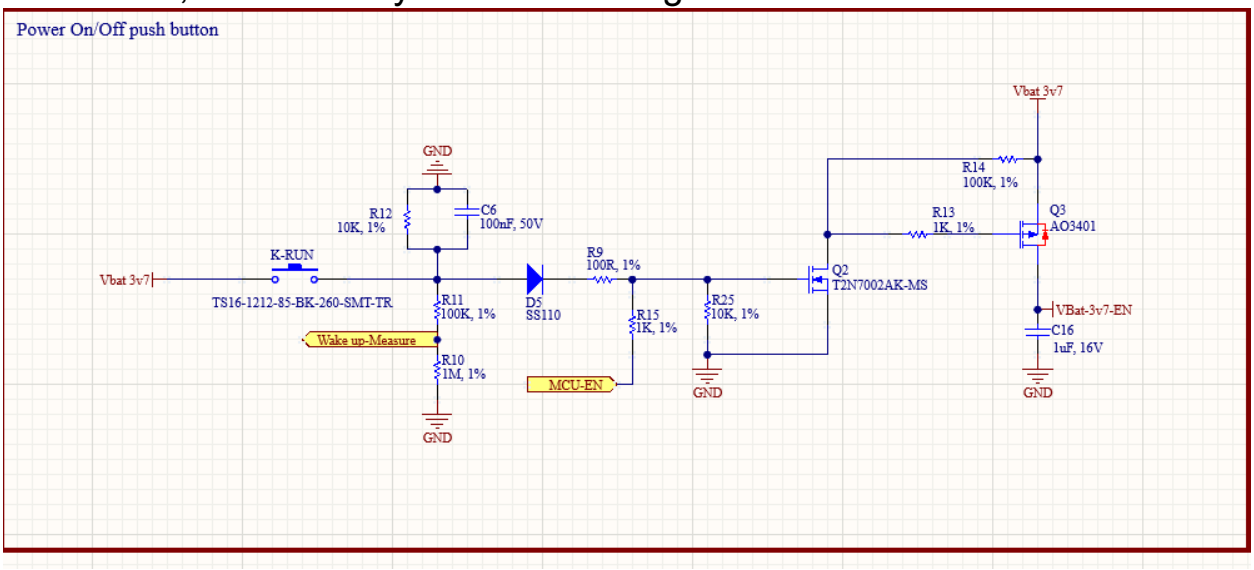
The battery voltage sensing circuit connects to the microcontroller's ADC pin. A voltage divider is used to step down the 3.7V supply to 3.3V; this ensures a 1:1 ratio relative to the microcontroller's reference, allowing a digital value of 4095 to be obtained when 3.3V is applied to the pin.

The resistors in this divider cannot have excessively high values, as this would prevent the signal from reaching the microcontroller; since the pin's current capacity is in the microampere range, very high resistance would fail to deliver sufficient current, rendering the circuit non-functional. Consequently, we selected resistor values of 1.5 kΩ and 13 kΩ for this stage

Fixing the problem of the device not turning on

Based on the explanation in the previous section, the resistive divider in the circuit below needs to have significantly lower values; the 1.5kΩ and 13kΩ resistors were selected for this purpose. Additionally, the values of resistors R14 and R25 were reduced. High resistance values had previously prevented sufficient current from flowing through the drain of Transistor 2, while lowering R25 reduced the input

resistance, thereby increasing the circuit current.



Fixing the problem of the device not turning off

Following recent modifications to the device's startup process, a malfunction occurred: the system would either freeze after the loading bar appeared or shut down before the loading bar even displayed.

After extensive hardware analysis—focusing on current supply and resistance adjustments—we traced the issue to the microcontroller's limited memory. The existing microcontroller had only 32 KB of memory, and we were utilizing 31.24 KB; even the slightest unnecessary memory usage triggered a stack overflow in the flash memory, causing the microcontroller to hang.

To resolve this, we first eliminated or downsized all unnecessary variables. We also removed I2C-related functions and files that were carried over from the previous version but served no purpose in the current implementation, thereby freeing up additional available memory.

Furthermore, given that new variables are now located at different addresses, it is necessary to place the non-volatile parameters—which we intend to store in the microcontroller's flash memory—at the appropriate address and page.

Since the microcontroller has limited free memory, a thorough analysis was required; to this end, I examined the generated .map file and selected a suitable area of available memory.

Fixed the issue of the device turning on unexpectedly

added a 100ms delay at the start of the code so that the device activates only if the button remains pressed for that duration—otherwise, the signal is treated as noise .and the device remains off

```
  HAL_Delay(100);
  if (HAL_GPIO_ReadPin(WakeUp_GPIO_Port, WakeUp_Pin)==GPIO_PIN_SET);
    else LATCH_OFF();
```

Fixing the zero-display issue during flow jumps from low to high values

This bug occurred when the device was connected to an oxygen concentrator and the flow changed abruptly. It appeared as though the sensor—which was continuously transmitting data to the microcontroller—was momentarily sending a zero value.

To address this, code was added to verify the duration of the zero reading: if the zero persisted for a few milliseconds, it would be displayed; otherwise, it would be treated as an outlier.

This logic is implemented within the `counterforbtn` timer interrupt to measure the duration of the zero state.

```
  if (ox_Flow>=200 || ox_Flow<0){
              ox_Flow=255-ox_Flow;}
  if(ox_Flow<3 && ox_Flow>0)      {
      FLowHighTime=0; FLowBlink=true;  FLowishigh=false; CountFlow=true; CounterFlowIsZero++;}
  else if (ox_Flow>=3  && ox_Flow<200 ){if (CountFlow==true){FLowHighTime++;if(FLowHighTime==10)CountFlow=false;} FLowishigh=true; FLowBlink=true; }
  else if (ox_Flow==0) CounterFlowIsZero++;

  if (btnPressed==1) {CounterForbtn++;}
```

And finally, if the value is not zero after this time, use the previous value instead of the flow value .

```
if ((flow_dec_prv-flow_dec>=2 || flow_dec - flow_dec_prv >=2) && flow_int>=1){
  if(flow_dec>0) {ssd1306_printFixed2_SPI_String_8_16(40,20,FlowFrame, STYLE_NORMAL,0);CounterFlowIsZero=0;}
  if (CounterFlowIsZero>5)  {
    ssd1306_printFixed2_SPI_String_8_16(40,20,FlowFrame, STYLE_NORMAL,0); CounterFlowIsZero=0;}

}
else if (flow_int<1 || JumpFromMenu==true){
    if(flow_dec>0) {ssd1306_printFixed2_SPI_String_8_16(40,20,FlowFrame, STYLE_NORMAL,0);CounterFlowIsZero=0;}
          if (CounterFlowIsZero>5)  {
            ssd1306_printFixed2_SPI_String_8_16(40,20,FlowFrame, STYLE_NORMAL,0); CounterFlowIsZero=0;}
JumpFromMenu=false;
}

flow_dec_prv=flow_dec;
```